

PostgreSQL Auditing with pgaudit

Table of Contents

1. [Learning Objectives](#)
 2. [Why Audit PostgreSQL](#)
 3. [Built-in PostgreSQL Logging](#)
 4. [pgaudit Extension](#)
 5. [Session Auditing](#)
 6. [Object Auditing](#)
 7. [Log Analysis and Queries](#)
 8. [Trigger-Based Auditing](#)
 9. [Event Triggers for DDL Auditing](#)
 10. [Centralized Audit Log Pattern](#)
 11. [Common Mistakes](#)
 12. [Best Practices](#)
 13. [Performance Considerations](#)
 14. [Interview Questions](#)
 15. [Exercises and Solutions](#)
-

Learning Objectives

By the end of this module, you will be able to:

- Configure pgaudit for comprehensive SQL auditing
 - Set up trigger-based auditing for table change history
 - Create an event trigger for DDL auditing
 - Query and analyze audit logs
 - Implement a centralized audit trail for compliance
 - Balance audit completeness with performance
-

Why Audit PostgreSQL

Auditing answers the question: **"Who did what, when, and to which data?"**

Use cases:

Compliance	GDPR, HIPAA, SOC2, PCI-DSS
Security	Detect unauthorized access
Forensics	Investigate breaches
Change Management	Track schema/data changes
Accountability	DBA activity monitoring

What to audit:

- Authentication events (login/logout)
- DDL: CREATE, ALTER, DROP tables, indexes, etc.
- DML on sensitive tables: INSERT, UPDATE, DELETE
- SELECT on sensitive data (optional, high volume)

- Role/privilege changes
- Configuration changes

Built-in PostgreSQL Logging

Before pgaudit, configure PostgreSQL's built-in logging for a baseline:

```
# postgresql.conf

#-----
# LOGGING SETTINGS
#-----

logging_collector = on
log_directory = '/var/log/postgresql'
log_filename = 'postgresql-%Y-%m-%d_%H%M%S.log'
log_rotation_age = 1d
log_rotation_size = 100MB
log_truncate_on_rotation = off

# What to log
log_connections = on           # Log new connections
log_disconnections = on       # Log disconnections
log_duration = off            # Don't log all query durations (high volume)
log_min_duration_statement = 1000 # Log queries taking > 1 second
log_checkpoints = on          # Log checkpoint activity
log_lock_waits = on           # Log lock wait events

# Log format
log_line_prefix = '%t [%p]: [%l-1] user=%u,db=%d,app=%a,client=%h '
# %t=timestamp, %p=PID, %l=log line number, %u=user, %d=db, %a=app, %h=client IP

# DDL logging (without pgaudit)
log_min_messages = info
log_statement = 'ddl'          # Log all DDL statements
# Options: 'none', 'ddl', 'mod' (ddl+INSERT/UPDATE/DELETE), 'all'
```

pgaudit Extension

pgaudit provides detailed, structured audit logging far beyond built-in logging.

Installation

```
# Install pgaudit (must match PostgreSQL major version)
# Ubuntu/Debian
sudo apt-get install -y postgresql-16-pgaudit

# RHEL/CentOS
sudo yum install -y pgaudit_16
```

```
# From source
git clone https://github.com/pgaudit/pgaudit.git
cd pgaudit
make install USE_PGXS=1 PG_CONFIG=/usr/lib/postgresql/16/bin/pg_config
```

```
# postgresql.conf
shared_preload_libraries = 'pgaudit' # Must add before other libraries

# After restart, create the extension:
# CREATE EXTENSION pgaudit;
```

```
# Restart PostgreSQL (required for shared_preload_libraries)
sudo systemctl restart postgresql@16-main
```

```
-- Create the extension
CREATE EXTENSION pgaudit;
```

pgaudit Configuration

```
# postgresql.conf – pgaudit settings

# Session-level auditing (all queries by the session)
pgaudit.log = 'ddl, write' # Audit these statement classes

# Classes available:
# read      : SELECT, COPY (reading from tables)
# write     : INSERT, UPDATE, DELETE, TRUNCATE, COPY (writing)
# function  : Function calls and DO blocks
# role      : GRANT, REVOKE, CREATE ROLE, ALTER ROLE, DROP ROLE
# ddl       : All DDL (CREATE, ALTER, DROP, TRUNCATE, COMMENT, etc.)
# misc      : SET, RESET, DISCARD, FETCH, MOVE, LISTEN, UNLISTEN, LOAD
# misc_set  : SET statements
# all       : All of the above (high volume!)

# Object-level auditing (audit specific tables/columns)
pgaudit.log_catalog = on      # Include catalog queries in audit
pgaudit.log_level = log       # Log level for audit messages
pgaudit.log_parameter = on    # Include query parameters in audit log
pgaudit.log_relation = on     # Include relation name for each statement
pgaudit.log_statement_once = off # Log only once per statement (not per relation)
```

Session Auditing

Session auditing logs ALL queries by all sessions (or sessions matching pg_hba.conf).

```
# Audit all write operations + DDL for all sessions
pgaudit.log = 'write, ddl'
```

```
# Audit ALL operations (high volume, use with care)
pgaudit.log = 'all'
```

```
-- Per-role auditing (audit specific users more aggressively)
ALTER ROLE dba_user SET pgaudit.log = 'all';
ALTER ROLE service_account SET pgaudit.log = 'write, role';
ALTER ROLE reporting_user SET pgaudit.log = 'read'; -- Audit reads too

-- Per-database auditing
ALTER DATABASE sensitive_db SET pgaudit.log = 'all';
```

Session Audit Log Format

```
2024-01-15 14:32:05 UTC [1234]: [1-1]
user=appuser,db=mydb,app=psql,client=192.168.1.50
LOG:  AUDIT: SESSION,1,1,WRITE,INSERT,TABLE,public.orders,
      "INSERT INTO orders (user_id, product_id, amount) VALUES (42, 100, 99.99)",
      <not logged>

Format: AUDIT: SESSION, audit_event_id, sub_id, class, command, object_type,
object_name, statement, parameter
```

Object Auditing

Object auditing is more precise — audit specific tables, functions, or schemas.

```
-- First: enable session auditing with no classes (just object auditing)
-- pgaudit.log = 'none' # or leave unset

-- Audit specific table (any access)
ALTER TABLE sensitive_data SET (pgaudit.log = 'all');
ALTER TABLE credit_cards SET (pgaudit.log = 'write');
ALTER TABLE user_passwords SET (pgaudit.log = 'read, write');

-- Object audit format includes relation name
-- LOG: AUDIT: OBJECT,1,1,READ,SELECT,TABLE,public.sensitive_data,"SELECT...",<not
logged>

-- Combined: session auditing for DDL + object auditing for specific tables
-- pgaudit.log = 'ddl, role' (in postgresql.conf)
-- ALTER TABLE pii_data SET (pgaudit.log = 'read, write'); (per table)
```

Log Analysis and Queries

Parsing pgaudit Log into Structured Data

```

-- Create a table to hold structured audit events
-- (Import via log shipping, Filebeat, etc.)
CREATE TABLE audit_log_raw (
    id BIGSERIAL PRIMARY KEY,
    log_time TIMESTAMPTZ,
    username TEXT,
    database TEXT,
    application TEXT,
    client_ip INET,
    audit_type TEXT,    -- SESSION or OBJECT
    class TEXT,        -- READ, WRITE, DDL, etc.
    command TEXT,      -- SELECT, INSERT, UPDATE, etc.
    object_type TEXT,  -- TABLE, VIEW, etc.
    object_name TEXT,  -- schema.table
    statement TEXT,
    parameters TEXT,
    ingested_at TIMESTAMPTZ DEFAULT now()
);

-- Create indexes for common query patterns
CREATE INDEX idx_audit_log_username ON audit_log_raw(username);
CREATE INDEX idx_audit_log_time ON audit_log_raw(log_time);
CREATE INDEX idx_audit_log_object ON audit_log_raw(object_name);
CREATE INDEX idx_audit_log_command ON audit_log_raw(command);

```

```

-- Audit analysis queries:

-- Who modified the orders table today?
SELECT username, client_ip, command, log_time
FROM audit_log_raw
WHERE object_name LIKE '%orders%'
    AND command IN ('INSERT', 'UPDATE', 'DELETE')
    AND log_time > current_date
ORDER BY log_time DESC;

-- DDL changes in the last 7 days
SELECT log_time, username, command, object_name, statement
FROM audit_log_raw
WHERE class = 'DDL'
    AND log_time > now() - INTERVAL '7 days'
ORDER BY log_time DESC;

-- User activity summary
SELECT username,
    COUNT(*) AS total_statements,
    COUNT(*) FILTER (WHERE command IN ('INSERT', 'UPDATE', 'DELETE')) AS writes,
    COUNT(*) FILTER (WHERE command = 'SELECT') AS reads,
    MIN(log_time) AS first_activity,
    MAX(log_time) AS last_activity
FROM audit_log_raw

```

```

WHERE log_time > now() - INTERVAL '30 days'
GROUP BY username
ORDER BY total_statements DESC;

-- Suspicious: large number of SELECT on sensitive table
SELECT username, COUNT(*) AS select_count, MIN(log_time) AS first, MAX(log_time) AS
last
FROM audit_log_raw
WHERE object_name = 'public.credit_cards'
AND command = 'SELECT'
AND log_time > now() - INTERVAL '1 day'
GROUP BY username
HAVING COUNT(*) > 100
ORDER BY select_count DESC;

-- Failed login attempts (from log_connections)
SELECT log_time, username, client_ip, statement
FROM audit_log_raw
WHERE class = 'AUTH' AND command = 'FAILED'
ORDER BY log_time DESC
LIMIT 50;

```

Trigger-Based Auditing

For row-level change history with before/after values:

```

-- Audit trail table
CREATE TABLE table_audit_log (
    id BIGSERIAL PRIMARY KEY,
    schema_name TEXT NOT NULL,
    table_name TEXT NOT NULL,
    operation CHAR(1) NOT NULL CHECK (operation IN ('I', 'U', 'D')),
    app_user_id INT, -- Application user (from SET LOCAL)
    db_user TEXT DEFAULT session_user,
    session_ip INET DEFAULT inet_client_addr(),
    old_data JSONB, -- Previous values (for UPDATE, DELETE)
    new_data JSONB, -- New values (for INSERT, UPDATE)
    changed_columns TEXT[], -- Only for UPDATE: which columns changed
    query TEXT, -- The SQL statement
    transaction_id BIGINT DEFAULT txid_current(),
    created_at TIMESTAMPTZ DEFAULT now()
);

-- Audit trigger function
CREATE OR REPLACE FUNCTION audit_trigger_function()
RETURNS TRIGGER AS $$
DECLARE
    audit_row table_audit_log%ROWTYPE;
    excluded_cols TEXT[] = ARRAY[]::TEXT[];
    new_json JSONB;

```

```

old_json JSONB;
BEGIN
    audit_row.schema_name = TG_TABLE_SCHEMA;
    audit_row.table_name = TG_TABLE_NAME;
    audit_row.app_user_id = NULLIF(current_setting('app.user_id', true), '')::INT;
    audit_row.db_user = session_user;
    audit_row.session_ip = inet_client_addr();
    audit_row.query = current_query();
    audit_row.transaction_id = txid_current();

    IF TG_ARGV[0] IS NOT NULL THEN
        excluded_cols = TG_ARGV[0]::TEXT[];
    END IF;

    IF (TG_OP = 'UPDATE') THEN
        audit_row.operation = 'U';
        old_json = row_to_json(OLD)::JSONB - excluded_cols;
        new_json = row_to_json(NEW)::JSONB - excluded_cols;
        audit_row.old_data = old_json;
        audit_row.new_data = new_json;
        -- Record only changed columns
        audit_row.changed_columns = ARRAY(
            SELECT key FROM jsonb_each_text(old_json) o
            WHERE old_json->key IS DISTINCT FROM new_json->key
        );
        IF audit_row.changed_columns = '{}'::TEXT[] THEN
            RETURN NULL; -- Nothing changed, don't audit
        END IF;

    ELSIF (TG_OP = 'DELETE') THEN
        audit_row.operation = 'D';
        audit_row.old_data = row_to_json(OLD)::JSONB - excluded_cols;

    ELSIF (TG_OP = 'INSERT') THEN
        audit_row.operation = 'I';
        audit_row.new_data = row_to_json(NEW)::JSONB - excluded_cols;
    END IF;

    INSERT INTO table_audit_log VALUES (audit_row.*);
    RETURN NULL; -- AFTER trigger, return value ignored
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

-- Attach trigger to tables to be audited
CREATE TRIGGER orders_audit
    AFTER INSERT OR UPDATE OR DELETE ON orders
    FOR EACH ROW EXECUTE FUNCTION audit_trigger_function('{}'); -- No excluded cols

-- Exclude sensitive columns from audit
CREATE TRIGGER users_audit

```

```
AFTER INSERT OR UPDATE OR DELETE ON users
FOR EACH ROW EXECUTE FUNCTION audit_trigger_function('{password_hash, ssn}');
```

Event Triggers for DDL Auditing

```
-- Capture all DDL changes (CREATE, ALTER, DROP)
CREATE TABLE ddl_audit_log (
    id BIGSERIAL PRIMARY KEY,
    command_tag TEXT,          -- CREATE TABLE, ALTER INDEX, etc.
    object_type TEXT,          -- table, index, sequence, etc.
    schema_name TEXT,
    object_identity TEXT,      -- full qualified name
    db_user TEXT,
    client_ip INET,
    query TEXT,
    created_at TIMESTAMPTZ DEFAULT now()
);

-- Event trigger function
CREATE OR REPLACE FUNCTION audit_ddl_commands()
RETURNS event_trigger AS $$
DECLARE
    obj record;
BEGIN
    FOR obj IN SELECT * FROM pg_event_trigger_ddl_commands() LOOP
        INSERT INTO ddl_audit_log(
            command_tag, object_type, schema_name, object_identity,
            db_user, client_ip, query
        ) VALUES (
            obj.command_tag,
            obj.object_type,
            obj.schema_name,
            obj.object_identity,
            session_user,
            inet_client_addr(),
            current_query()
        );
    END LOOP;
END;
$$ LANGUAGE plpgsql;

-- Create the event trigger
CREATE EVENT TRIGGER audit_ddl
ON ddl_command_end
EXECUTE FUNCTION audit_ddl_commands();

-- Also audit DROP events
CREATE OR REPLACE FUNCTION audit_drop_commands()
RETURNS event_trigger AS $$
DECLARE
```



```

    obj record;
BEGIN
    FOR obj IN SELECT * FROM pg_event_trigger_dropped_objects() LOOP
        INSERT INTO ddl_audit_log(command_tag, object_type, schema_name,
object_identity, db_user, query)
            VALUES ('DROP', obj.object_type, obj.schema_name, obj.object_identity,
session_user, current_query());
        END LOOP;
END;
$$ LANGUAGE plpgsql;

CREATE EVENT TRIGGER audit_drops
    ON sql_drop
    EXECUTE FUNCTION audit_drop_commands();

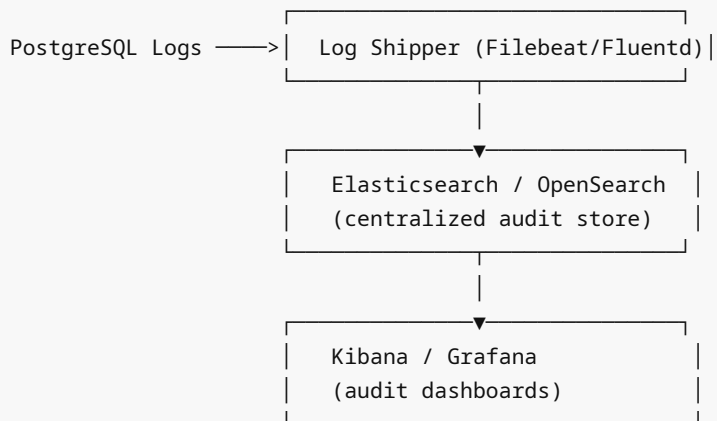
-- Test
CREATE TABLE test_audit_table (id INT);
-- Check: SELECT * FROM ddl_audit_log ORDER BY created_at DESC LIMIT 5;

DROP TABLE test_audit_table;
-- Check: SELECT * FROM ddl_audit_log WHERE command_tag = 'DROP' ORDER BY created_at
DESC LIMIT 5;

```

Centralized Audit Log Pattern

Architecture:



OR: Direct table-based audit (within PostgreSQL)

PostgreSQL pgaudit → CSV log file → PostgreSQL audit_log table (via COPY)

Common Mistakes

1. **Auditing everything with `pgaudit.log = all`** — massive log volume degrades performance
2. **Not auditing DDL** — schema changes go unnoticed
3. **Storing audit logs in the same database** — admin can DELETE audit records
4. **No index on audit log timestamp** — slow queries on large audit tables

5. **Logging passwords/secrets in parameters** — `pgaudit.log_parameter = on` logs query params
 6. **Not monitoring audit log growth** — disk fill from unrotated audit logs
 7. **Audit triggers excluded from backups** — audit log not in backup
 8. **Not testing audit completeness** — "blind spots" in coverage
 9. **Single audit trail for both app and DBA** — harder to analyze separately
 10. **No alerting on suspicious patterns** — audit logs not being reviewed
-

Best Practices

1. **Separate audit storage** — write audit logs to a separate, write-once location
 2. **Audit at minimum: DDL + role changes + writes on sensitive tables**
 3. **Use pgaudit object auditing** for targeted, lower-volume audit of specific tables
 4. **Index audit tables** on time, user, table_name for fast queries
 5. **Partition audit tables** by month for performance and retention management
 6. **Set retention policy** — compliance may require 1-7 years
 7. **Alert on suspicious patterns** — large SELECT count, off-hours access, privilege escalation
 8. **Test audit coverage** — verify that audited actions appear in logs
 9. **Protect audit logs** — only audit reader can query; no updates/deletes
 10. **Ship logs externally** — don't rely solely on PostgreSQL for audit storage
-

Performance Considerations

```
# Minimize pgaudit overhead:
pgaudit.log = 'ddl, write'      # NOT 'all' - avoids logging every SELECT
pgaudit.log_catalog = off      # Don't audit catalog queries (very high volume)
pgaudit.log_parameter = off    # Don't log parameters (can contain sensitive data +
                                large)
pgaudit.log_statement_once = on # Log statement once, not for every relation
                                touched
```

```
-- Partition audit table for performance
CREATE TABLE table_audit_log_2024_01 PARTITION OF table_audit_log
FOR VALUES FROM ('2024-01-01') TO ('2024-02-01');

-- Automatic partitioning with pg_partman extension
-- Retention: DROP old partitions after retention period
```

Interview Questions

Q1: What is pgaudit and why is it better than `log_statement = 'all'` ?

A: `log_statement = 'all'` logs all SQL statements but without structure — it's hard to parse and analyze. pgaudit provides structured audit records with fields for audit type, command class, object type, and object name. It distinguishes between DDL/DML/reads/writes and can audit specific tables (object auditing). pgaudit integrates with standard log analysis tools and produces compliance-grade audit trails.

Q2: What is the difference between session auditing and object auditing in pgaudit?

A: Session auditing logs all statements by all sessions matching the `pgaudit.log` configuration. It's coarse-grained — either all DDL, all writes, etc. Object auditing logs statements that affect specific database objects (tables, functions, etc.) regardless of session settings. Object auditing is set per-table: `ALTER TABLE sensitive_data SET (pgaudit.log = 'all')`. You can combine both.

Q3: How would you implement row-level change auditing?

A: Create an audit table storing `old_data` (JSONB), `new_data` (JSONB), `operation` (I/U/D), `user`, `timestamp`, and `query`. Create a trigger function using `AFTER INSERT OR UPDATE OR DELETE FOR EACH ROW` that captures `row_to_json(OLD)` and `row_to_json(NEW)` and inserts into the audit table. For `UPDATE`, compute which columns changed. Use `SECURITY DEFINER` to prevent the triggering user from bypassing audit writes.

Q4: How do you audit DDL changes (CREATE TABLE, ALTER TABLE)?

A: Use PostgreSQL Event Triggers: (1) Create a function that calls `pg_event_trigger_ddl_commands()` and inserts into an audit table. (2) Create an event trigger with `ON ddl_command_end`. (3) For `DROP` events, create another trigger with `ON sql_drop` using `pg_event_trigger_dropped_objects()`. Event triggers fire for all DDL including changes made by superusers.

Q5: How would you detect unauthorized access attempts from audit logs?

A: Query the audit log for: (1) Failed authentication in `log_connections` output. (2) Access outside business hours: `WHERE EXTRACT(HOUR FROM log_time) NOT BETWEEN 8 AND 18`. (3) Unusual `SELECT` volume on sensitive tables. (4) Access from unexpected IP addresses. (5) Privilege escalation (`SET ROLE`, `GRANT`). Automate with a cron job that runs these queries and sends alerts.

Q6: How do you protect audit logs from tampering by DBA?

A: (1) Ship logs to an external, append-only log system immediately (Elasticsearch, S3, Splunk) via Filebeat/Fluentd. (2) Store audit in a separate database with restricted write access. (3) Use `GRANT INSERT ONLY` (not `UPDATE/DELETE`) on audit tables. (4) Hash log entries with a chained hash (each entry includes hash of previous). (5) Use immutable log storage (WORM — Write Once Read Many).

Q7: What is the performance impact of trigger-based auditing?

A: Significant for high-write tables. Each `INSERT/UPDATE/DELETE` triggers an audit `INSERT`, roughly doubling write I/O. Mitigations: (1) Only audit tables that truly need it. (2) Make audit `INSERT` asynchronous (`LISTEN/NOTIFY` + background worker). (3) Write audit data to an unlogged table first, then move to permanent storage. (4) Partition audit tables so queries don't scan everything.

Exercises and Solutions

Exercise 1: Deploy pgaudit with Compliance Configuration

```
# postgresql.conf for SOC2 compliance
shared_preload_libraries = 'pgaudit'

# Audit DDL (schema changes) and role changes everywhere
pgaudit.log = 'ddl, role'

# Specific databases audit more aggressively
# (Set via ALTER DATABASE)
```

```
-- Enable per-database aggressive auditing
ALTER DATABASE production_db SET pgaudit.log = 'ddl, write, role';
ALTER DATABASE staging_db SET pgaudit.log = 'ddl';

-- For DBA roles: audit everything they do
ALTER ROLE dbadmin SET pgaudit.log = 'all';
ALTER ROLE readonly_user SET pgaudit.log = 'read';

-- Sensitive tables: full object audit
ALTER TABLE payments SET (pgaudit.log = 'all');
ALTER TABLE user_credentials SET (pgaudit.log = 'all');
ALTER TABLE audit_log SET (pgaudit.log = 'all');
```

Exercise 2: Query Audit Log for Suspicious Activity

```
-- Detect: access to sensitive tables outside business hours
SELECT
    log_time,
    username,
    client_ip,
    command,
    object_name
FROM audit_log_raw
WHERE object_name IN ('public.payments', 'public.user_credentials')
    AND (
        EXTRACT(DOW FROM log_time) IN (0, 6) -- Weekend
        OR EXTRACT(HOUR FROM log_time) NOT BETWEEN 7 AND 20 -- Off-hours
    )
    AND log_time > now() - INTERVAL '7 days'
ORDER BY log_time DESC;
```

Cross-References

- [01 authentication methods.md](#) — Auditing connection events
- [02 roles and privileges.md](#) — Auditing privilege changes
- [03 row level security.md](#) — Auditing RLS-protected tables
- [08 compliance considerations.md](#) — Compliance audit requirements